

Chapitre 6:

Compilation et édition des liens

Cours 9: Les étapes de la compilation

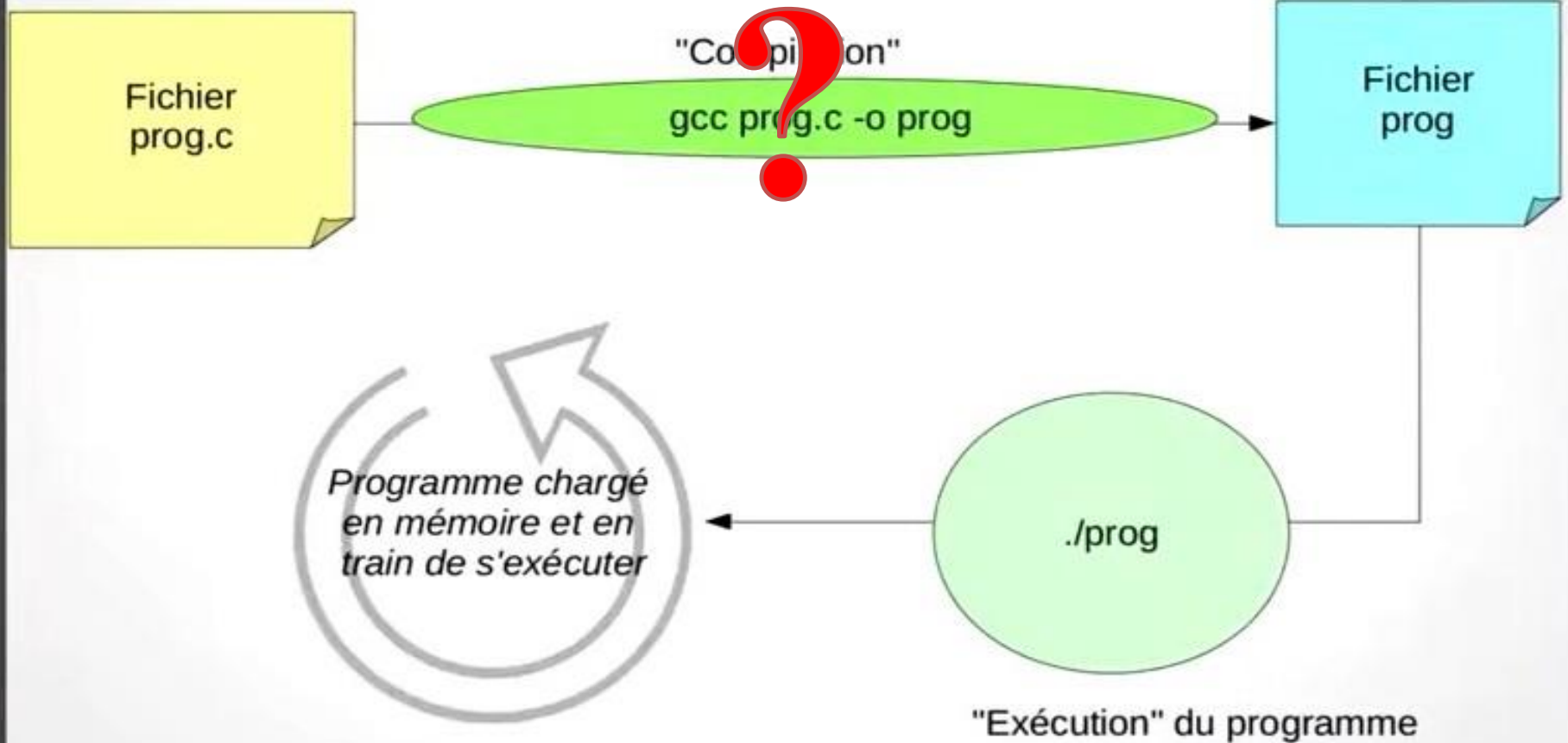
Plan

- 1) Introduction**
- 2) Pré-traitement**
- 3) Compilation**
- 4) Assemblage**
- 5) Édition de liens**
- 6) Exécution**
- 7) Récapitulation**

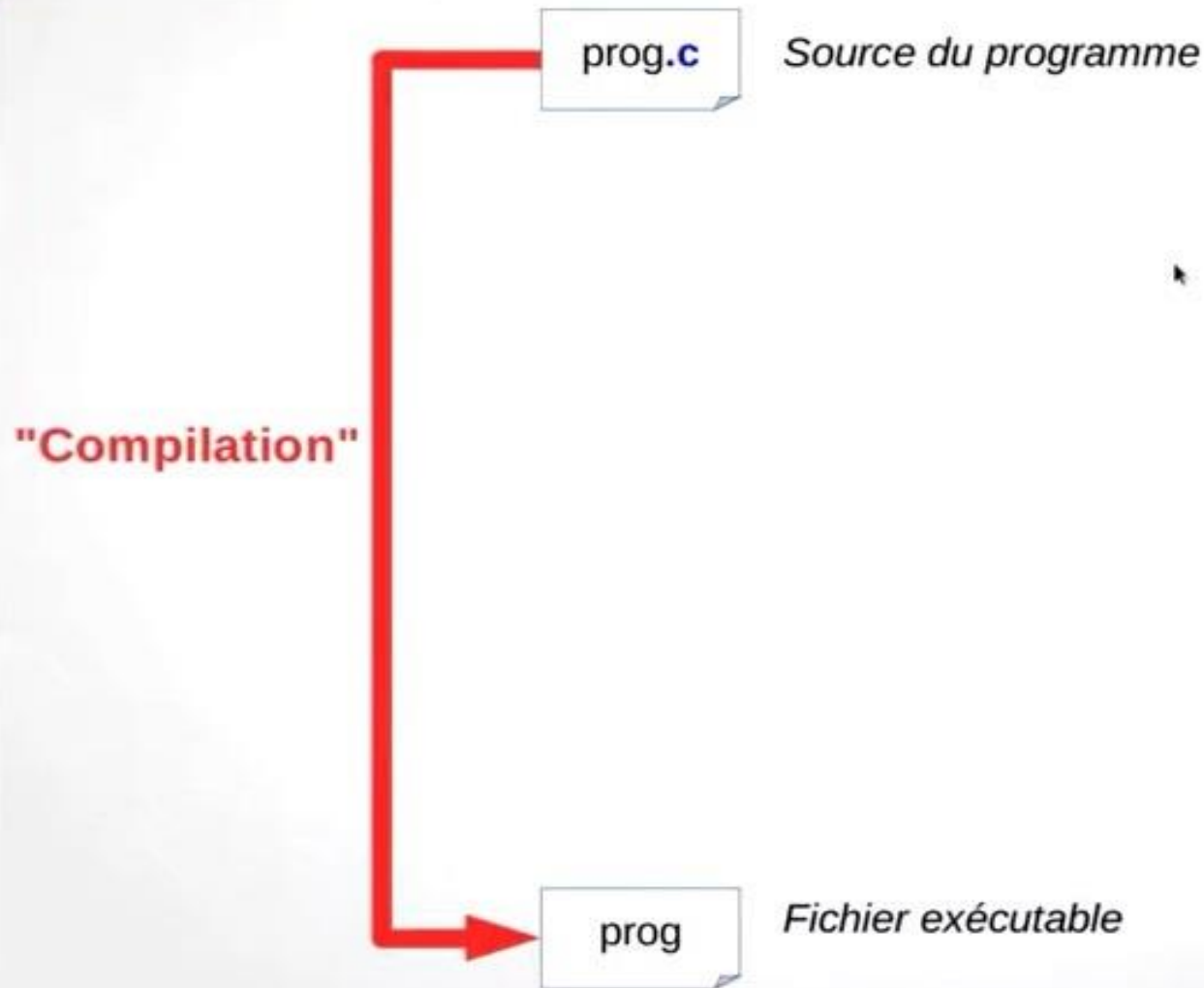
Plan

- 1) Introduction**
- 2) Pré-traitement
- 3) Compilation
- 4) Assemblage
- 5) Édition de liens
- 6) Exécution
- 7) Récapitulation

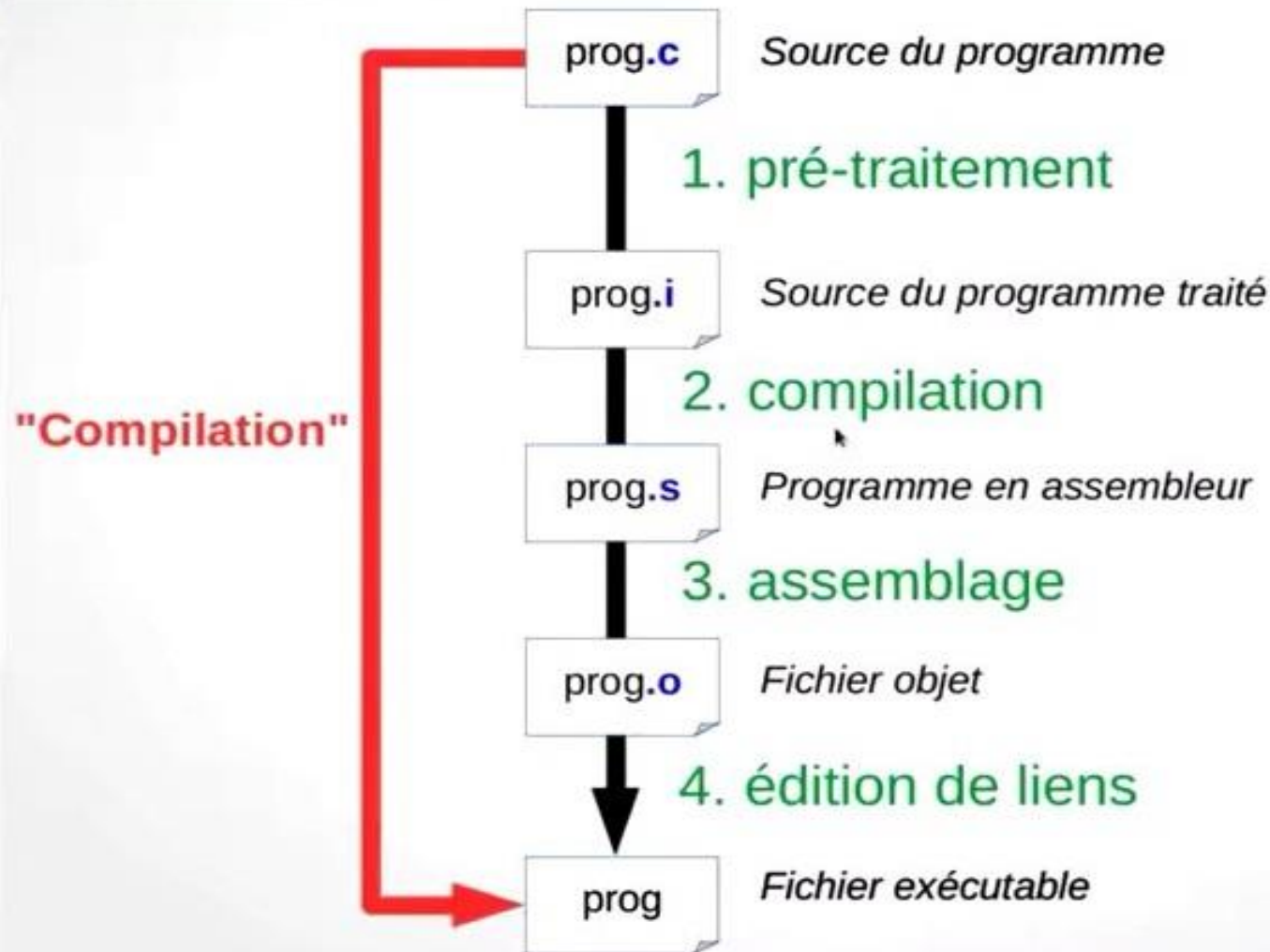
Compiler ? Exécuter ?



Compilation ? compilation ?



Compilation ? compilation ?



Le code source d'un programme

Fichier prog.c

```
#define MAX 8
const char* chaine = "bonjour" ;
int nb = MAX ;
int                                x ;

...fonction_0 (...) ;
...fonction_1 (...) ;
...fonction_2 (...) ;

/* programme principal */
int main () {
    ... fonction_1...chaine...
    ...x ...fonction_0...nb...
}
...fonction_0 (...) {
    ...nb...fonction_2...
}
...fonction_1 (...) {
    ...chaine...
}
...fonction_2 (...) {
    ... // un commentaire utile
}
```

Plan

- 1) Introduction
- 2) Pré-traitement**
- 3) Compilation
- 4) Assemblage
- 5) Édition de liens
- 6) Exécution
- 7) Récapitulation

1. Pré-traitement (pre-processing)

- enlève les commentaires
 - tout ce qui est entre `/*` et `*/`
 - le reste de la ligne après `//`
- raccorde les lignes et sépare les tokens par un seul espace insecable
- évalue les directives de pré-traitement (signalé par le caractère `#`)
 - **#include** inclus les fichiers d'en-tête
 - **#define** remplace les macros par leurs valeurs
 - **#ifdef**, **#ifndef**, **#if defined**, **#else**, **#undef...#endif** evaluation conditionnelle des directives de pré-traitement
- **cpp** fichier.c > fichier.i
- ou directement depuis le fichier C
- **gcc -E** fichier.c > fichier.i
- Le fichier résultant porte souvent l'extension **.i**

Le code source d'un programme

```
#define MAX 8
const char* chaine = "bonjour" ;
int nb = MAX ;
int x ;
...fonction_0 (...);
...fonction_1 (...);
...fonction_2 (...);
/* programme principal */
int main () {
    ... fonction_1...chaine...
    ...x ...fonction_0...nb...
}
...fonction_0 (...) {
    ...nb...fonction_2...
}
...fonction_1 (...) {
    ...chaine...
}
...fonction_2 (...) {
    ... // un commentaire utile
}
```

cpp prog.c > prog.i

```
# 1 "prog.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "<command-line>" 2
# 1 "prog.c"
const char* chaine = "bonjour" ;
int nb = 8 ;
int x ;
...fonction_0 (...);
...fonction_1 (...);
...fonction_2 (...);
int main () {
    ... fonction_1...chaine...
    ...x ...fonction_0...nb...
}
...fonction_0 (...) {
    ...nb...fonction_2...
}
...fonction_1 (...) {
    ...chaine...
}
...fonction_2 (...) {
    ...
}
```

Pré-traitement (Preprocessing)

Exercice:

Que devient ce programme source après pré-traitement ?

Code source

```
#include "header1.h"
#define PI 3.14
#define fois2(a) a+a

// Fonction principale
int main(){
    int val=fois2(PI);
#ifdef PLUS5
    val+= 5;
#else
    val-= 5;
#endif
}
```

Code "pré-traité"

```
...Code du "header1.h"...

int main(){
int val=3.14+3.14;
val-= 5;
}
```

Plan

- 1) Introduction
- 2) Pré-traitement
- 3) Compilation**
- 4) Assemblage
- 5) Édition de liens
- 6) Exécution
- 7) Récapitulation

2. Compilation

Le code source du programme pré-traité est transformé en langage assembleur propre à l'architecture de la machine cible (x86, PowerPC, ARM, MIPS, ...)

cc1 *prog.i* **-o** *prog.s*

ou directement depuis le fichier c

gcc -S *prog.c* **-o** *prog.s*

Le fichier résultant porte souvent l'extension **.s**

Au cas où aucun nom de fichier d'exécutable n'aurait été précisé avec **-o**, l'exécutable généré a par défaut le nom du fichier suivi de **.s**

Compilation

```
const char* chaine = "bonjour" ;
int nb = 8 ;
int x ;
```

```
int main () {
    ... fonction_1...chaine...
    ...x ...fonction_0...nb...
}
...fonction_0 (...) {
    ...nb...fonction_2...
}
...fonction_1 (...) {
    ...chaine...
}
...fonction_2 (...) {
    ...
}
}
```

cc1 prog.i -o prog.s

```
.file "prog.c"
.globl chaine
.section .rodata

.LC0:
.string "bonjour"
.data
.align 8
.type chaine, @object
.size chaine, 8

chaine:
.quad .LC0
.globl nb
.align 4
.type nb, @object
.size nb, 4

nb:
.long 8
.comm x,4,4
.text
.globl main
.type main, @function

main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $0, %eax
call fonction_0
movl $4, nb(%rip)
movl $0, %eax
call fonction_1
movl x(%rip), %eax
addl $1, %eax
movl %eax, x(%rip)
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE0:
.size main, -main
.globl fonction_0
.type fonction_0, @function
```

```
fonction_0:
.LFB1:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $8, nb(%rip)
movl $0, %eax
call fonction_2
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE1:
.size fonction_0, -fonction_0
.globl fonction_1
.type fonction_1, @function

fonction_1:
.LFB2:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE2:
.size fonction_1, -fonction_1
.globl fonction_2
.type fonction_2, @function

fonction_2:
.LFB3:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE3:
.size fonction_2, -fonction_2
.ident "GCC: (Debian 4.9.2-10) 4.9.2"
.section .note.GNU-stack,"",@progbits
```

Compilation (Optimisation)

L'objectif est d'améliorer les performances du programme (vitesse d'exécution et usage mémoire).

Techniques principales :

- **Inlining de fonctions:** suppression des appels de fonctions (-finline-functions)
- **Suppression du code mort:** fonctions et variables inutilisées
- **Optimisation des boucles:** déroulement, déplacement des invariants (-funroll-loops)
- **Optimisation pour le processeur cible :** (-march=...)

Remarque importante :

- ✗ Perte de portabilité (code dépendant de l'architecture et du processeur)
- ✓ Gain significatif en performance

Plan

- 1) Introduction
- 2) Pré-traitement
- 3) Compilation
- 4) Assemblage**
- 5) Édition de liens
- 6) Exécution
- 7) Récapitulation

3. Assemblage

Le code assembleur est ensuite assemblé pour générer du code machine.

Le fichier produit est appelé fichier objet.

as *prog.s* **-o** *prog.o*

ou directement depuis le fichier .c

gcc -c *prog.c* **-o** *prog.o*

Le fichier résultant porte souvent l'extension **.o**

Au cas où aucun nom de fichier d'exécutable n'aurait été précisé avec **-o**, l'exécutable généré a par défaut le nom le nom du fichier suivi de **.o**

Assemblage

Fichier objet (.o) contient :

- Le code machine généré à partir de l'assembleur
- La table des symboles

Symbole :

- Nom d'une fonction ou d'une variable
- Défini dans un fichier .c et utilisable par d'autres fichiers
- Un fichier peut utiliser des symboles externes

Table des symboles :

- Enregistre chaque identificateur avec :
 - type, emplacement mémoire, portée
- Pour les fonctions :
 - nombre et types d'arguments
 - mode de passage
 - valeur de retour

Assemblage

Exercice:

Quels sont les symboles exportés et les symboles importés pour ce deux fichiers ?

fichier1.c

```
int a = 0 ;  
void fct1()  
{  
  Printf("Hello");  
}
```

symboles fichier1.c

Symboles exportés :

a
fct1

Symboles importés :

fichier2.c

```
void main()  
{  
  fct1() ;  
  a++ ;  
}
```

symboles fichier2.c

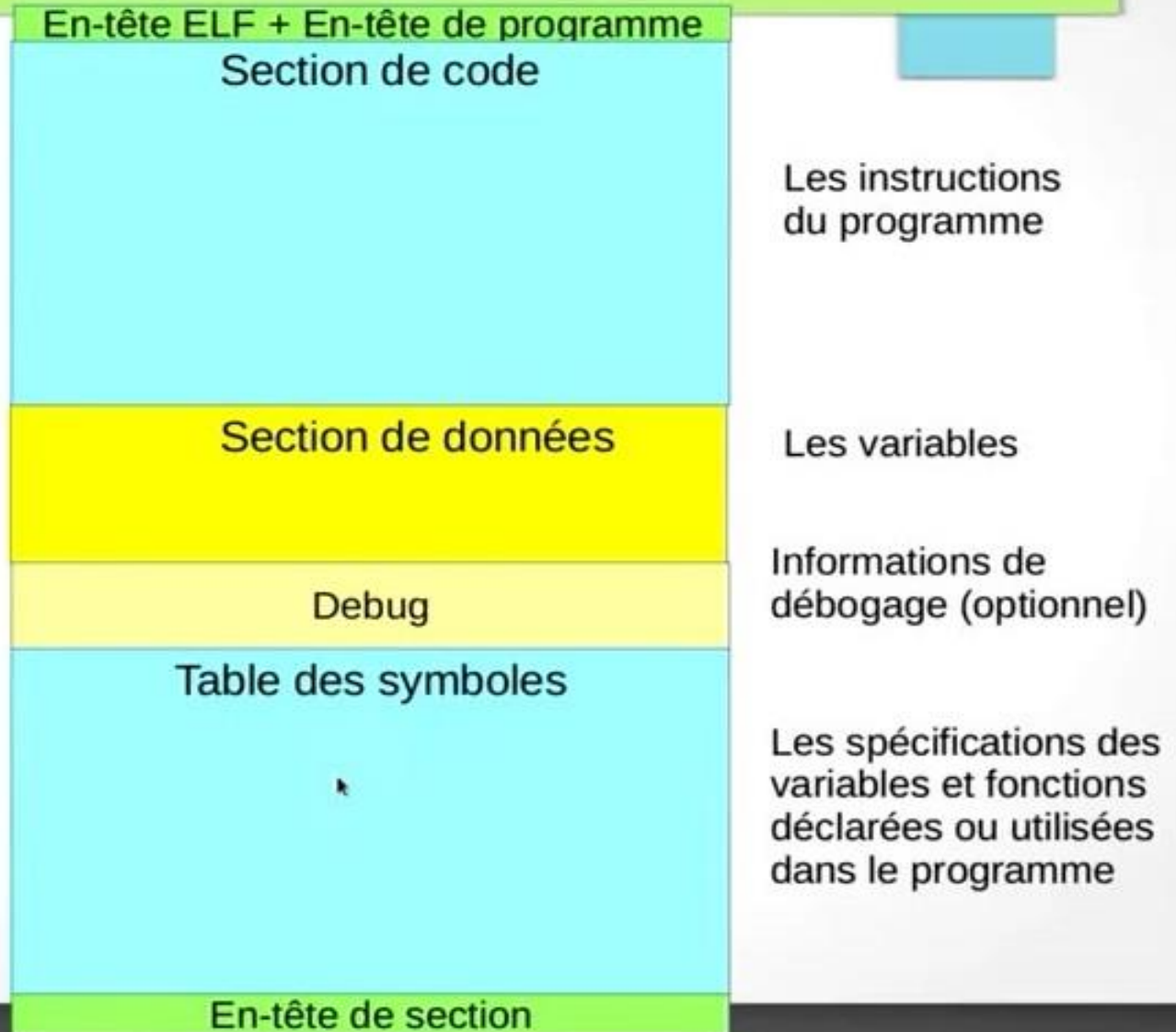
Symboles exportés :

main

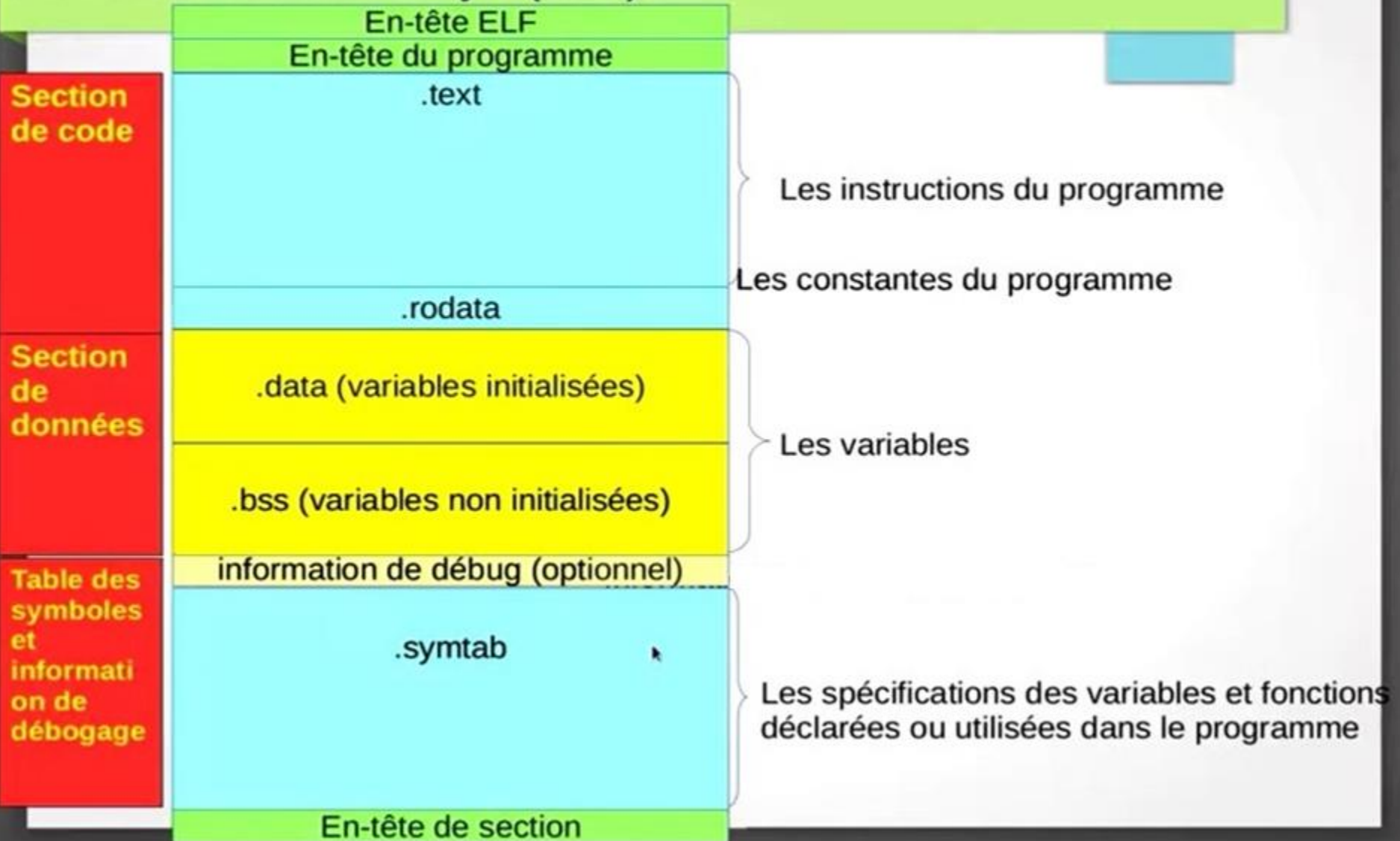
Symboles importés :

fct1
a

Représentation simplifiée d'un programme



Format d'un fichier objet (ELF)



Assemblage

```

file      "prog.c"
globl     chaine
section   .rodata

LCO:
string    "bonjour"
data
align 8
type      chaine, @object
size      chaine, 8

chaine:
quad      LCO
globl     nb
align 4
type      nb, @object
size      nb, 4

nb:
long      8
comm      x, 4, 4
text
globl     main
type      main, @function

main:
LFB0:
.cfi_startproc
pushq     %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq      %rsp, %rbp
.cfi_def_cfa_register 6
movl      $0, %eax
call      fonction_0
movl      $4, nb(%rip)
movl      $0, %eax
call      fonction_1
movl      x(%rip), %eax
addl      $1, %eax
movl      %eax, x(%rip)
popq      %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE0:
.size      main, -main
globl     fonction_0
type      fonction_0, @function

fonction_0:
LFB1:
.cfi_startproc
pushq     %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq      %rsp, %rbp
.cfi_def_cfa_register 6
movl      $8, nb(%rip)
movl      $0, %eax
call      fonction_2
popq      %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE1:
.size      fonction_0, -fonction_0
globl     fonction_1
type      fonction_1, @function

fonction_1:
LFB2:
.cfi_startproc
pushq     %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq      %rsp, %rbp
.cfi_def_cfa_register 6
popq      %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE2:
.size      fonction_1, -fonction_1
globl     fonction_2
type      fonction_2, @function

fonction_2:
LFB3:
.cfi_startproc
pushq     %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq      %rsp, %rbp
.cfi_def_cfa_register 6
popq      %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE3:
.size      fonction_2, -fonction_2
.ident    "GCC: (Debian 4.9.2-10) 4.9.2"
.section  .note.GNU-stack,"",@progbits

```

Assemblage

as *prog.s -o prog.o*

```
.file "prog.c"
.globl _start
.section .text
.LC0:
.string "bonjour"
.data
.align 8
.type "chaine", @object
.size "chaine", 8
.chaine:
.quad .LC0
.globl nb
.align 4
.type "nb", @object
.size "nb", 4
.nb:
.long 8
.comm "nb", 4, 4
.text
.globl main
.type "main", @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $0, %eax
call fonction_0
movl $4, %eax
call fonction_1
movl x(%rip), %eax
addl $1, %eax
movl %eax, x(%rip)
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.globl fonction_0
.type fonction_0, @function
```

```
fonction_0:
.LFB1:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $8, %eax
movl $0, %eax
call fonction_2
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE1:
.size fonction_0, .-fonction_0
.globl fonction_1
.type fonction_1, @function
fonction_1:
.LFB2:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE2:
.size fonction_1, .-fonction_1
.globl fonction_2
.type fonction_2, @function
fonction_2:
.LFB3:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE3:
.size fonction_2, .-fonction_2
.ident "GCC: (Debian 4.9.2-10) 4.9.2"
.section .note.GNU-stack,"",@progbits
```

En-tête ELF + En-tête de programme

Section de code

Section de données

Debug

Table des symboles

En-tête de section

Assemblage

as *prog.s -o prog.o*

En-tête ELF + En-tête de programme

```

file      prog.c
globl     chaine
section   .rodata

.LC0:
.string   "bonjour"

chaine:
.type     chaine, @object
.size     chaine, 8

quad      .LC0
globl     nb
align 4
.type     nb, @object
.size     nb, 4

nb:
.long     8
.comm     4, 4
.text
globl     main
.type     main, @function

fonction_0:
.LFB1:
.cfi_startproc
pushq    %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq     %rsp, %rbp
.cfi_def_cfa_register 6
movl     $8, nb(%rip)
movl     $0, %eax
call     fonction_2
popq     %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE1:
.size     fonction_0, -fonction_0
globl     fonction_1
.type     fonction_1, @function

fonction_1:
.LFB2:
.cfi_startproc
pushq    %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq     %rsp, %rbp
.cfi_def_cfa_register 6
popq     %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE2:
.size     fonction_1, -fonction_1
globl     fonction_2
.type     fonction_2, @function

fonction_2:
.LFB3:
.cfi_startproc
pushq    %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq     %rsp, %rbp
.cfi_def_cfa_register 6
popq     %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

LFE3:
.size     fonction_2, -fonction_2
.ident   "GCC: (Debian 4.9.2-10) 4.9.2"
.section .note.GNU-stack,"",@progbits

LFE0:
.size     main, -main
globl     fonction_0
.type     fonction_0, @function
    
```

Section de code			
0x000			
0x02D	main:	0x15D	fonction_1:
0x02E	.LFB0:	0x15E	.LFB2:
...	pushq %rbp	...	.cfi_startproc
...	call fonction_0	...	pushq %rbp
...	movl \$4, nb(%rip)	...	.cfi_offset 6, -16
...	call fonction_1	...	
...	movl x(%rip), %eax	0x178	fonction_2:
...		0x178	.LFB3:
0x105	fonction_0:	...	.cfi_startproc
.LFB1:		...	pushq %rbp
...	.cfi_def_cfa_register 6	...	
...	movl \$0, %eax	0x2BA	.LC0:
...	call fonction_2	...	.string "bonjour"
...	popq %rbp	...	.size chaine, 8
...			
Section de données			
0x455			
0x456			
0x457			
...			
Debug			
...			
Table des symboles			
0xE38			
0xE3A			
0xE3C			
0xE3F			
0xE40			
0xE45			
...			
En-tête de section			

Assemblage

as prog.s -o prog.o

```
.file "prog.c"
.globl chaîne
.section .rodata

.LC0:
.string "bonjour"

.type chaîne, @object
.size chaîne, 8

.quad .LC0
.align 4
.type nb, @object
.size nb, 4

.long 8
.comm 4, 4
.text
.globl main
.type main, @function

.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $0, %eax
call fonction_0
movl $4, nb(%rip)
movl $0, %eax
call fonction_1
movl x(%rip), %eax
addl $1, %eax
movl %eax, x(%rip)
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE0:
.size main, .-main
.globl fonction_0
.type fonction_0, @function

fonction_0:
.LFB1:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $8, nb(%rip)
movl $0, %eax
call fonction_2
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE1:
.size fonction_0, .-fonction_0
.globl fonction_1
.type fonction_1, @function

fonction_1:
.LFB2:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE2:
.size fonction_1, .-fonction_1
.globl fonction_2
.type fonction_2, @function

fonction_2:
.LFB3:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE3:
.size fonction_2, .-fonction_2
.ident "GCC: (Debian 4.9.2-10) 4.9.2"
.section .note.GNU-stack,"",@progbits
```

En-tête ELF + En-tête de programme

Section de code	
0x000	
0x02D	main:
0x02E	.LFB0: pushq %rbp
...	call fonction_0
...	movl \$4, nb(%rip)
...	call fonction_1
...	movl x(%rip), %eax
0x105	fonction_0:
...	.LFB1: .cfi_def_cfa_register 6
...	movl \$0, %eax
...	call fonction_2
...	popq %rbp
0x15D	fonction_1:
0x15E	.LFB2: .cfi_startproc
...	pushq %rbp
...	.cfi_offset 6, -16
0x178	fonction_2:
0x178	.LFB3: .cfi_startproc
...	pushq %rbp
0x2BA	.LC0: .string "bonjour"
...	.size chaîne, 8
Section de données	
0x455	nb: .long 8;
0x456	x: .type @object #0x2A
0x457	chaîne: .type @object #0x2A
...	
Debug	
...	
Table des symboles	
0xE38	
0xE3A	
0xE3C	
0xE3F	
0xE40	
0xE45	
...	
En-tête de section	

Assemblage

as *prog.s -o prog.o*

En-tête ELF + En-tête de programme

Section de code

```

0x000
.....
0x02D  main:
0x02E  .LFB0:
        pushq %rbp
        call  fonction_0
        movl  $4, nb(%rip)
        call  fonction_1
        movl  x(%rip), %eax
.....
0x105  fonction_0:
.LFB1:
        cfi_def_cfa_register 6
        movl  $0, %eax
        call  fonction_2
        popq  %rbp
.....
0x15D  fonction_1:
0x15E  .LFB2:
        cfi_startproc
        pushq %rbp
        cfi_offset 6, -16
        cfi_def_cfa_register 6
        pushq %rbp
        call  fonction_0
        movl  $4, nb(%rip)
        call  fonction_1
        movl  x(%rip), %eax
.....
0x178  fonction_2:
0x178  .LFB3:
        cfi_startproc
        pushq %rbp
        cfi_offset 6, -16
        cfi_def_cfa_register 6
        movl  $0, %eax
        call  fonction_2
        popq  %rbp
.....
0x2BA  .LC0:
        .string "bonjour"
        .size  chaîne, 8
    
```

Section de données

```

.....
0x455  nb
0x456  x: .long 8 ;
0x457  chaîne: .type @object #0x2A
    
```

Debug

Table des symboles

```

.....
0xE38  nb
.....
0xE3A  x
.....
0xE3C  chaîne
.....
0xE3F  main
.....
0xE3F  fonction_0
.....
0xE40  fonction_1
.....
0xE45  fonction_2
    
```

En-tête de section

```

.file "prog.c"
.globl chaîne
.section .rodata
.LC0:
.string "bonjour"

.type chaîne, @object
.size chaîne, 8

chaîne:
.quad .LC0
.globl nb
.align 4
.type nb, @object
.size nb, 4

nb:
.long 8
.comm x, 4, 4
.type x, @object
.globl main
.type main, @function

.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $0, %eax
call fonction_0
movl $4, nb(%rip)
movl $0, %eax
call fonction_1
movl x(%rip), %eax
addl $1, %eax
movl %eax, x(%rip)
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE0:
.size main, -main
.globl fonction_0
.type fonction_0, @function

fonction_0:
.LFB1:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE1:
.size fonction_0, -fonction_0
.globl fonction_1
.type fonction_1, @function

fonction_1:
.LFB2:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE2:
.size fonction_1, -fonction_1
.globl fonction_2
.type fonction_2, @function

fonction_2:
.LFB3:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE3:
.size fonction_2, -fonction_2
.ident "GCC: (Debian 4.9.2-10) 4.9.2"
.section .note.GNU-stack,"",@progbits
    
```

```
as prog.S -o prog.O
```

Section de code

```

function_0:
.LFB1:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    $8, %nb(%rip)
        movl    $0, %eax
        call    function_2
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc

function_1:
        .size    function_0, -function_0
        .globl   function_1
        .type    function_1, @function

function_1:
.LFB2:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc

.LFE2:
        .size    function_1, -function_1
        .globl   function_2
        .type    function_2, @function

function_2:
.LFB3:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc

.LFE3:
        .size    function_2, -function_2
        .ident    "GCC: (Debian 4.9.2-10) 4.9.2"
        section   .note.gnu-stack, @progbits

```

Section de données

Debug

0xE38	nb	#0x455
...		
0xE3A	x	#0x456
...		
0xE3C	chaîne	#0x457
...		
0xE3F	main	#0x02D
...		
0xE40	fonction_0	#0x105
...		
0xE41	fonction_1	#0x15D
...		
0xE45	fonction_2	#0x178

En-tête de section

Assemblage

Commandes pour visualiser la table des symboles:

- `objdump -t file.o`
- `nm file.o`
- `readelf -s file.o`

Plan

- 1) Introduction
- 2) Pré-traitement
- 3) Compilation
- 4) Assemblage
- 5) Édition de liens**
- 6) Exécution
- 7) Récapitulation

Edition de liens

L'édition de liens prend un ensemble de fichiers objets pour produire un programme exécutable.

- rassemble les fichiers objets du programme
- rassemble les fichiers objets du "système"
- extrait les fonctions des bibliothèques
 - les bibliothèques statiques
 - les bibliothèques dynamiques
- résout les problèmes d'adressage

ld -e point_d_entree prog.o -o prog

ou directement depuis les fichiers .c

gcc prog.c -o prog

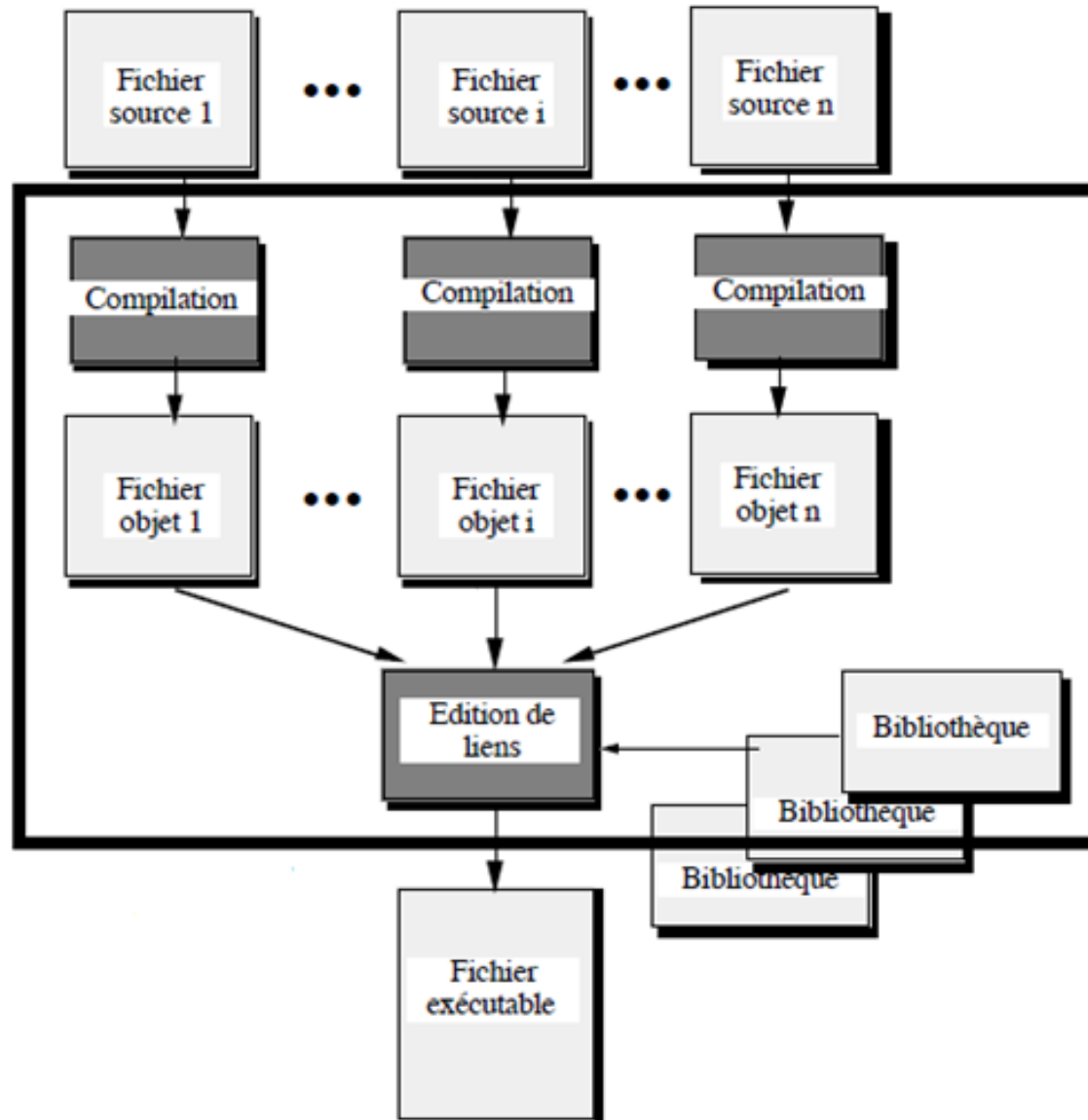
ou depuis les fichiers .o

gcc prog.o -o prog

Le fichier exécutable résultant ne porte souvent pas d'extension.

Au cas où aucun nom de fichier d'exécutable n'aurait été précisé avec **-o**, l'exécutable généré a par défaut le nom **a.out** (même si c'est un fichier au format ELF)

Édition des liens



Edition de liens I

En-tête ELF + En-tête de programme

Section de code

```
0x000
...
0x02D main:
0x02E .LFB0:
...
    pushq   %rbp
    call    fonction_0
...
    movl    $4, nb(%rip)
    call    fonction_1
...
    movl    x(%rip), %eax
...
0x105 fonction_0:
.LFB1:
...
    .cfi_def_cfa_register 6
    movl    $0, %eax
    call    fonction_2
...
    popq    %rbp
...
0x15D fonction_1:
0x15E .LFB2:
...
    .cfi_startproc
    pushq   %rbp
...
    .cfi_offset 6, -16
...
0x178 fonction_2:
0x178 .LFB3:
...
    .cfi_startproc
    pushq   %rbp
...
0x2BA .LC0:
    .string "bonjour"
    .size   chaine, 8
```

Section de données

```
...
0x455 nb
0x456 x: .long 8
0x457 chaine
```

ld -e main prog.o -o prog

Debug

Table des symboles

```
...
0xE38 nb          #0x455
0xE3A x           #0x456
0xE3C chaine      #0x457
0xE3E main        #0x02D
0xE3F fonction_0  #0x105
0xE40 fonction_1  #0x15D
0xE45 fonction_2  #0x178
...
```

En-tête de section

Edition de liens I

ld -e main prog.o -o prog

En-tête **_start #0x02D** : programme

Section de code

```
0x000
...
0x02D main:                                0x15D fonction_1:
0x02E .LFB0:                                0x15E .LFB2:
...                                         .cfi_startproc
...      pushq    %rbp                      pushq    %rbp
...      call     fonction_0                .cfi_offset 6, -16
...      movl     $4, nb(%rip)              ...
...      call     fonction_1                0x178 fonction_2:
...      movl     x(%rip), %eax             0x178 .LFB3:
...                                         .cfi_startproc
...                                         pushq    %rbp
...
0x105 fonction_0:                            ...
.LFB1:                                         0x2BA .LC0:
...                                         .string  "bonjour"
...      .cfi_def_cfa_register 6             .size    chaine, 6
...      movl     $0, %eax
...      call     fonction_2
...      popq     %rbp
...
```

Section de données

```
...
0x455 nb
0x456 x: .long 8 ;
0x457 chaine: .type @object #0x2A
...
```

Debug

Table des symboles

```
...
0xE38 nb                                #0x455
0xE3A x                                  #0x456
...
0xE3C chaine                             #0x457
0xE3F main                               #0x02D
...
0xE40 fonction_0                         #0x105
0xE40 fonction_1                         #0x15D
...
0xE45 fonction_2                         #0x178
...
```

En-tête de section

Plan

- 1) Introduction
- 2) Pré-traitement
- 3) Compilation
- 4) Assemblage
- 5) Édition de liens
- 6) Exécution**
- 7) Récapitulation

Programme exécutable

En-tête `_start #0x02D` e programme

Section de code

```
0x000
....
0x02D main:
0x02E .LFB0:
....
    pushq   %rbp
    call    fonction_0
    movl    $4, nb(%rip)
    call    fonction_1
    movl    x(%rip), %eax
....
0x105 fonction_0:
.LFB1:
....
    .cfi_def_cfa_register 6
    movl    $0, %eax
    call    fonction_2
    popq    %rbp
....
0x15D fonction_1:
0x15E .LFB2:
....
    .cfi_startproc
    pushq   %rbp
    .cfi_offset 6, -16
....
0x178 fonction_2:
.LFB3:
....
    .cfi_startproc
    pushq   %rbp
....
0x2BA .LC0:
    .string "bonjour"
    .size   chaine, 8
....
```

Section de données

```
....
0x455 nb
0x456 x: .long 8 ;
0x457 chaine: .type @object #0x2A
....
```

Debug

Table des symboles

```
....
0xE38 nb           #0x455
....
0xE3A x           #0x456
....
0xE3C chaine       #0x457
0xE3C main         #0x02D
....
0xE3F fonction_0   #0x105
....
0xE40 fonction_1   #0x15D
....
0xE45 fonction_2   #0x178
....
```

En-tête de section

toto.txt

prog

toto.c

toto.o

L'exécution du programme

En-tête ELF + En-tête de programme

0x000

Section de code

```

0x02D main:
0x02E .LFB0:
    pushq   %rbp
    call    fonction_0
    movl    $4, nb(%rip)
    call    fonction_1
    movl    x(%rip), %eax

0x105 fonction_0:
.LFB1:
    .cfi_def_cfa_register 6
    movl    $0, %eax
    call    fonction_2
    popq    %rbp

0x15D fonction_1:
0x15E .LFB2:
    .cfi_startproc
    pushq   %rbp
    .cfi_offset 6, -16

0x178 fonction_2:
0x178 .LFB3:
    .cfi_startproc
    pushq   %rbp

0x2BA .LC0:
    .string "bonjour"
    size    chaine, 8
    
```

Section de données

```

0x455 nb
0x456 x: .long 8 ;
0x457 chaine: .type @object #0x2A
    
```

Debug

Table des symboles

```

0xE38 nb           #0x455
0xE3A x           #0x456
0xE3C chaine      #0x457
0xE3F main        #0x02D
0xE40 fonction_0  #0x105
0xE41 fonction_1  #0x15D
0xE42 fonction_2  #0x178
    
```

En-tête de section

Segment de mémoire
(lecture seule)

Segment de mémoire
(en lecture écriture)

Segment de
mémoire Mappé

En-tête ELF

En-tête du programme

segment de code (.text)

.rodata

.data

.bss

Tas

(allocation mémoire dynamique)

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

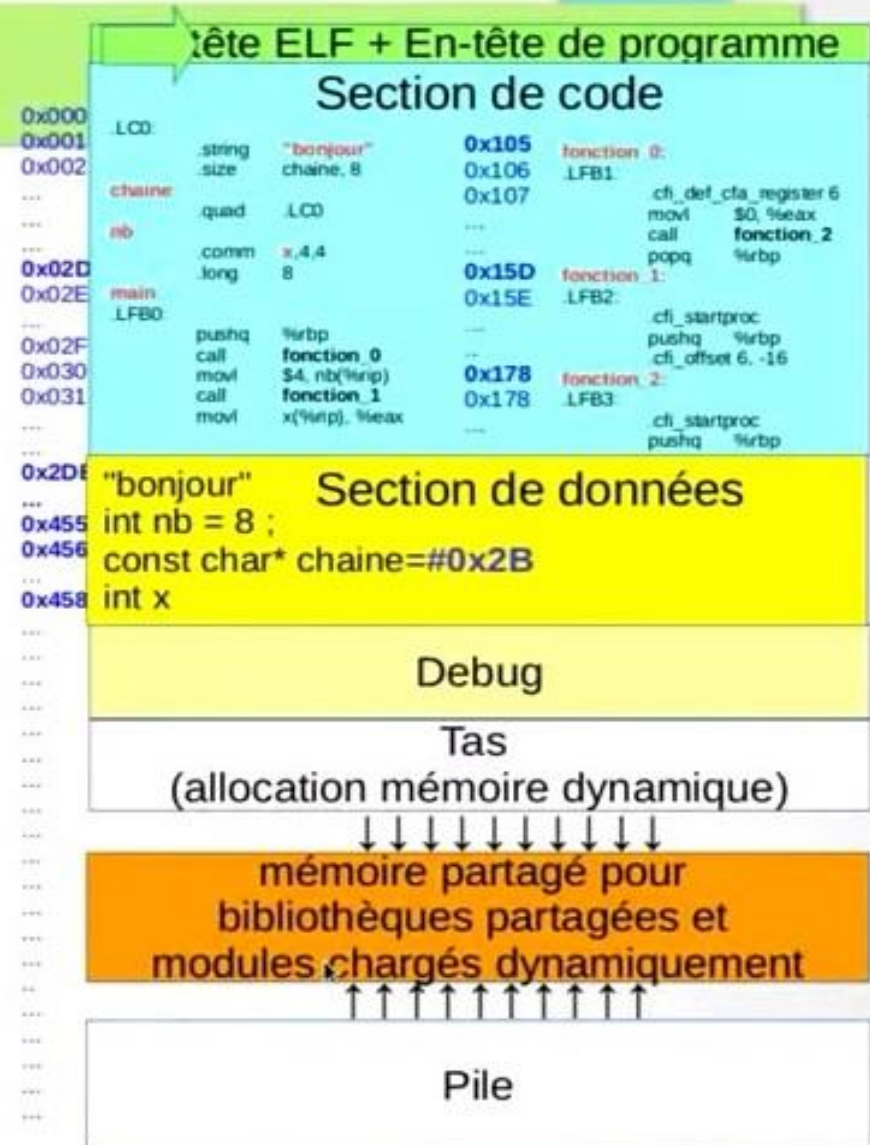
mémoire partagé pour
bibliothèques partagées et
modules chargés dynamiquement

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

Pile

En-tête de section

Exécution



Exécution



Tiens, et si j'exécutais mon programme ?



En-tête ELF + En-tête de programme

Section de code

0x000	.LC0:				
0x001		string	"bonjour"	0x105	fonction 0:
0x002		size	chaine, 8	0x106	.LFB1:
...	chaine	quad	.LC0	0x107	.cfi_def_cfa_register 6
...				...	movl \$0, %eax
...	nb	comm	x, 4, 4	...	call fonction_2
0x02D		long	8	0x15D	popq %rbp
0x02E	main			0x15E	.LFB2:
...	.LFB0:	pushq	%rbp	...	.cfi_startproc
0x02F		call	fonction_0	...	pushq %rbp
0x030		movl	\$4, nb(%rip)	0x178	.cfi_offset 6, -16
0x031		call	fonction_1	0x178	fonction 2:
...		movl	x(%rip), %eax	...	.LFB3:
...				...	.cfi_startproc
...				...	pushq %rbp

Section de données

```

0x2D6 "bonjour"
0x455 int nb = 8 ;
0x456 const char* chaine=#0x2B
0x458 int x
    
```

Debug

Tas
(allocation mémoire dynamique)

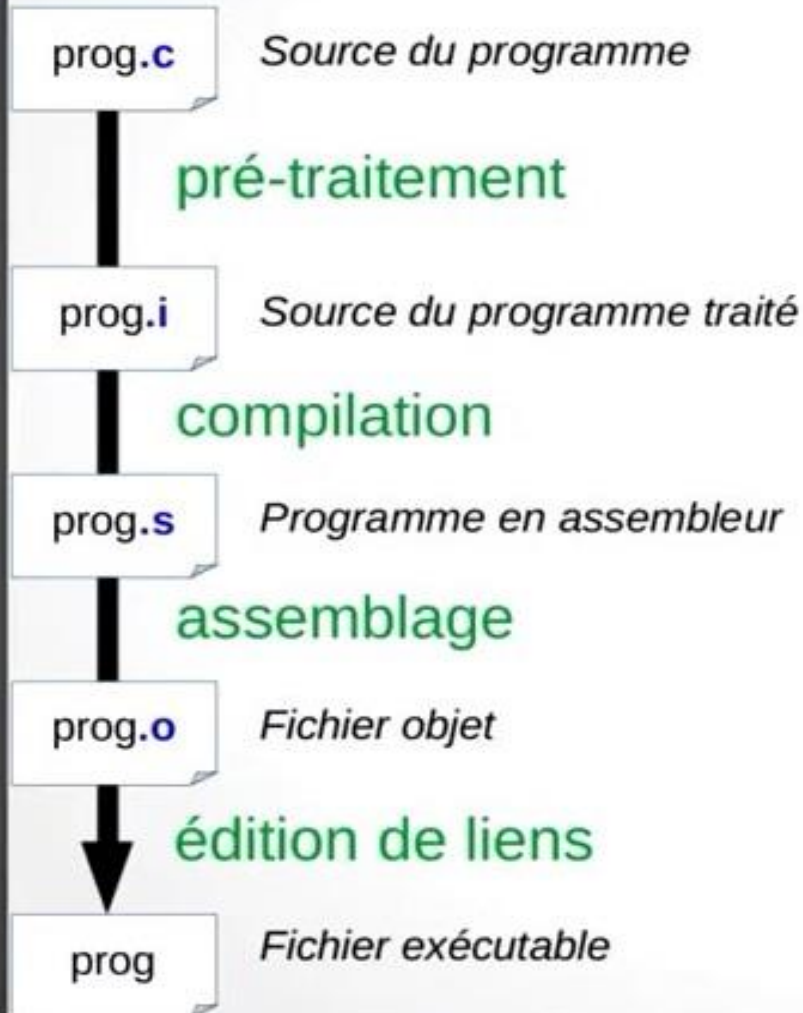
mémoire partagée pour
bibliothèques partagées et
modules chargés dynamiquement

Pile

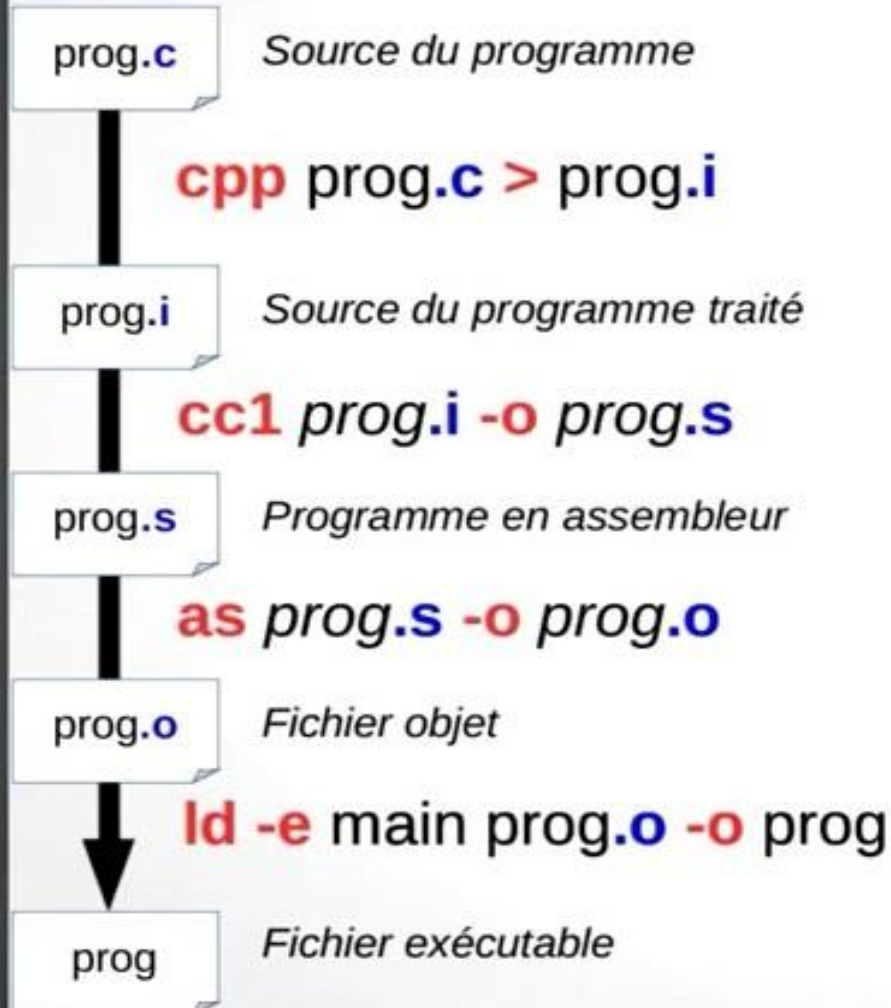
Plan

- 1) Introduction
- 2) Pré-traitement
- 3) Compilation
- 4) Assemblage
- 5) Édition de liens
- 6) Exécution
- 7) Récapitulation**

Etapes de la compilation



Etapes de la compilation



- 1)préprocesseur : **cpp**
- 2)compilateur : **cc1**
- 3)assembleur : **as**
- 4)éditeur de liens : **ld**

Vous n'avez pas à apprendre ces commandes par coeur : gcc les appelle lui-même

Etapes de la compilation

prog.c

gcc -E prog.c > prog.i

prog.i

gcc -S prog.i -o prog.s

prog.s

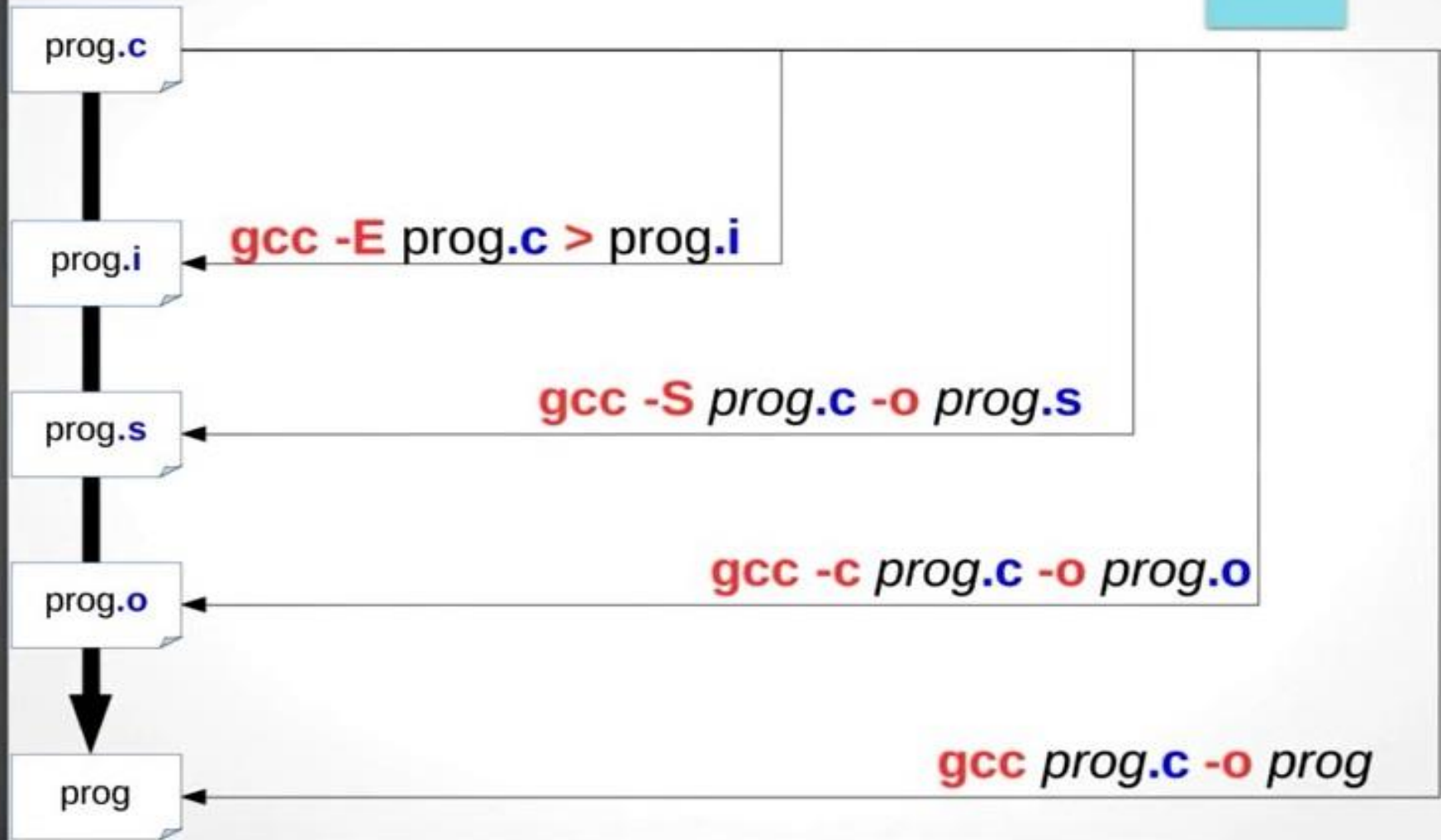
gcc -c prog.s -o prog.o

prog.o

gcc prog.o -o prog

prog

Etapes de la compilation



Etapes de la compilation

